

8. Contadores

Los contadores serán en cierto modo la primera aplicación que veremos de los registros puesto que para contar es necesario poder almacenar el valor actual de contaje para saber cuál será el siguiente valor.

Los contadores no se limitan a un problema académico pues su uso está muy extendido en aplicaciones reales. Sin ir más lejos podemos encontrar contadores en todos los procesadores puesto que estos ejecutan instrucciones siguiendo una cierta secuencia que habitualmente viene especificada por un registro en el que se acumula un número que indica la instrucción actual. Así por ejemplo empezamos por la instrucción 1, posteriormente viene la 2, luego la 3, luego la 4, luego quizás tenemos que saltar a la 8 ante una condición que hemos programado para continuar después con la instrucción 9, luego la 10, etc.

También los podemos encontrar en el diseño de aplicaciones específicas como por ejemplo relojes digitales puesto que después del minuto 1, vendrá el minuto 2, luego el 3, etc. y así sucesivamente hasta el minuto 59, que posteriormente pasará de nuevo al minuto 0 reiniciando el contaje.

En resumen, si observamos nuestro entorno seguro que podemos hallar muchas aplicaciones para las que se requiere un elemento que cuente, es decir un contador. En este tema veremos sus posibles variantes y cómo se puede diseñar un cierto contador a partir de sus especificaciones.

8.1. Introducción a los contadores

Un contador es un sistema secuencial con una serie de estados que indican un valor de contaje. Así pues, el contador dispondrá de una entrada de sincronismo que a medida que vaya recibiendo pulsos hará que el contador actualice su valor.

De este modo la estructura elemental de un contador sería la que se muestra en la siguiente figura, formada por un registro que almacenará el valor actual de contaje y un sistema combinacional cuya finalidad es determinar en su salida cuál será el siguiente valor de contaje a partir del valor actual. De esta manera, cuando llega un sincronismo el registro se actualiza con el nuevo valor y este valor, aplicado a la entrada del sistema combinacional, hará que éste último ponga a la entrada del contador el siguiente valor de contaje. Y así sucesivamente.

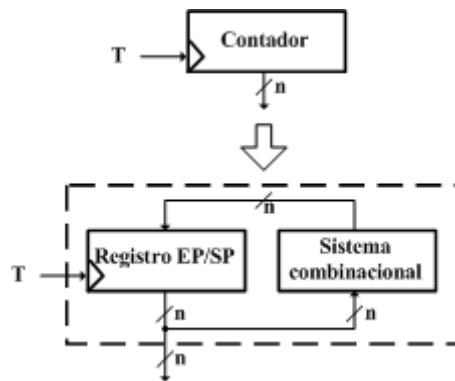


Figura 8.1. Esquema elemental de un contador formado por un registro EP/SP y un sistema combinacional. El registro almacena el valor actual de conteo mientras que el sistema combinacional se diseña para que en su salida se indique el próximo valor de conteo teniendo en su entrada el valor actual. Así, por ejemplo, si el valor actual es el 2, el sistema combinacional dará como salida el número 3 para que cuando llegue un sincronismo el registro se actualice con este valor. Posteriormente el sistema combinacional, ante un 3 de entrada, dará como salida un 4 para que cuando llegue un nuevo sincronismo el registro se actualice con este nuevo valor.

La implementación de los contadores se puede realizar de otras maneras. Así, por ejemplo, existen contadores asíncronos en los que el registro está implementado mediante biestables que no comparten la misma señal de sincronismo, lo que permite minimizar el número de sincronismos necesarios para provocar cambios en las salidas. Un contador asíncrono tiene un sistema combinacional más simple que uno síncrono, pero por otro lado los bits de salida cambian en distintos instantes de tiempo y como resultado podemos tener valores de conteo incorrectos hasta que las salidas se estabilizan.

En cualquier caso, en este tema nos centraremos sólo en el estudio de contadores síncronos.

8.1.1. Parámetros de diseño

Cuando se diseña un contador hay que tener en consideración una serie de parámetros de diseño puesto que afectan a su comportamiento.

- **Código de conteo.** Es la secuencia de valores que sigue el contador a medida que va recibiendo sincronismos. Por defecto los contadores cuentan en binario natural, por lo que si, por ejemplo, tenemos un contador de 3 bits, éste seguirá la secuencia 000, 001, 010, 011, 100, 101, 110 y 111; siendo éste su código de conteo. Podemos tener no obstante contadores que sigan otro código de conteo, como por ejemplo un Gray o bien incluso un código propio. Dentro del código de conteo además hay que diferenciar entre las siguientes opciones:
 - **Contador cíclico.** Cuando el contador alcanza la última combinación del código de conteo y llega un pulso de sincronismo entonces el contador pasa a la primera combinación del código, es decir “da la vuelta” al código y vuelve a empezar. Así, si el contador es un contador binario natural de 3 bits, veremos cómo después de la combinación 111 el contador volverá a la 000 y desde ésta seguirá contando.

- **Contador no cíclico.** Cuando el contador alcanza la última combinación del código de contaje y llega un pulso de sincronismo entonces el contador no pasa a la primera combinación del código, existiendo dos opciones:
 - **Contador no cíclico anclado.** El contador una vez alcanzada la última combinación mantendrá este valor por mucho que lleguen nuevos sincronismos. En el ejemplo anterior se mantendrá en la combinación 111.
 - **Contador no cíclico no anclado.** El contador una vez alcanzada la última combinación del código pasará a cualquier combinación no especificada cuando llegue un nuevo sincronismo. En el ejemplo anterior no podemos afirmar a qué combinación pasará el contador después de la combinación 111.
- **Capacidad de contaje.** Es el número de pulsos máximo que puede recibir para recorrer completamente el código de contaje. Así podemos tener un contador de 3 bits completo en tanto que cuenta desde la combinación 000 hasta la 111, o bien podría ser incompleto en tanto que sólo cuente hasta una combinación intermedia. La capacidad de contaje depende en gran medida del número de bits y el código utilizado. Así, un contador BCD, por mucho que tenga 4 bits de salida, sólo contará desde 0000 hasta 1001.
- **Frecuencia de contaje.** Especifica la máxima frecuencia a la que podemos aplicar los pulsos de sincronismo. En caso de superarla el contador no responderá correctamente y no actualizará las salidas a la misma frecuencia que recibe los pulsos.

8.1.2. Señales de entrada

Un contador puede presentar entradas adicionales al sincronismo para realizar acciones complementarias al simple contaje incremental. En la siguiente figura podemos ver algunas de estas señales.

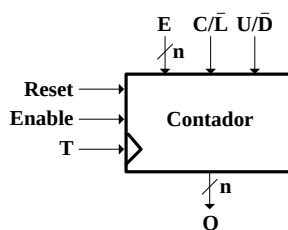


Figura 8.2. Contador con distintas señales de entrada que permiten controlar su comportamiento.

- **Enable.** Habilita el resto de entradas del contador de tal manera que, por ejemplo, si la entrada de *enable* está activa, cuando llega un pulso el contador pasa al siguiente valor del código de contaje. En caso de no estar habilitado no variará su salida por mucho que lleguen sincronismos ni se active ninguna otra entrada del contador. Puede ser activa por uno o bien por cero.

- **Reset.** Cuando se activa, la salida del contador adopta el valor correspondiente a la primera combinación del código de contaje. Es importante remarcar que la primera combinación del código de contaje no forzosamente se corresponde a todos los bits puestos a cero. Así, si por ejemplo tenemos un contador de 3 bits que cuyo código de contaje es el 010, 011, 110 y 101, entonces al recibir un *Reset* sus salidas adoptarán los valores correspondientes a la primera combinación, es decir 010. Esta señal puede tener dos modos de funcionamiento:
 - Síncrono, cuando llegue un pulso por la entrada de sincronismo *T*.
 - Asíncrono, en el mismo instante en el que se activa la señal *Reset*.
- **$C/\bar{L}$ o bien *Count/Load*.** Señal de control que hace que el dispositivo funcione como contador o bien que efectúe una carga en paralelo. El funcionamiento como contador es el visto hasta el momento. En cambio si está puesto en modo de carga paralelo entonces el registro interno adoptará el valor de la entrada externa *E*. Esta carga en paralelo puede responder a dos modos de funcionamiento distintos como en el caso anterior.
 - Síncrono, cuando llegue un pulso por la entrada de sincronismo *T*.
 - Asíncrono, en el mismo instante en el que se activa la señal $C/\bar{L}$.
- **$U/\bar{D}$ o bien *Up/Down*.** Señal de control que especifica si el contador cuenta en sentido ascendente o bien en sentido descendente, según el código de contaje. Su comportamiento sólo tiene sentido de manera síncrona.

Ejemplo 8.1

Para comprender mejor el comportamiento de los contadores veamos una evolución a lo largo del tiempo de un contador cíclico de 3 bits binario natural con las señales de entrada adicionales que acabamos de ver. Supondremos que la señal de *Reset* es síncrona y que $C/\bar{L}$ también es síncrona.

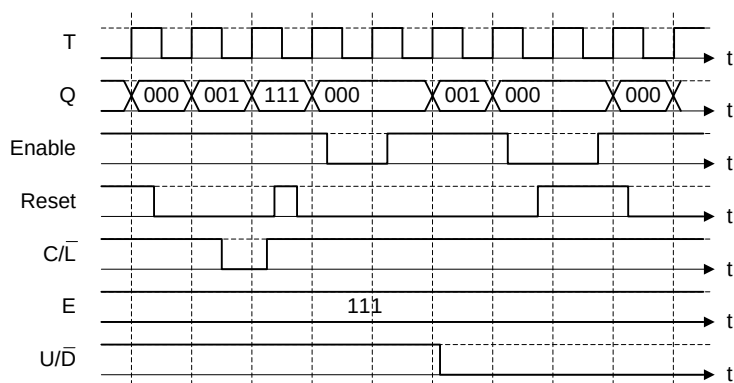


Figura 8.3. Cronograma de un contador cíclico binario natural de 3 bits. En el primer flanco el contador se actualiza a 000 al tener activada la señal de *Reset*. En el segundo flanco el contador cuenta ($C/\bar{L} = 1$) en modo ascendente ($U/\bar{D} = 1$). En el tercer flanco el contador carga el valor de la entrada externa *E* puesto que ($C/\bar{L} = 0$). En el cuarto flanco el contador cuenta en modo ascendente y al ser cíclico y estar en el último valor vuelve a iniciarse a 000. En el quinto flanco el contador no varía de valor puesto que no está habilitado por *Enable*. En el sexto flanco el contador vuelve a contar de manera ascendente hasta 001. En el séptimo flanco el contador cuenta de manera descendente volviendo a 000. En el octavo flanco el contador no está habilitado y no cambia de valor. En el noveno flanco el contador está reseteado por lo que se pone a 000 (de hecho, ya estaba en este valor). En el décimo flanco el contador descenderá a 111 puesto que cuenta en modo descendente.

Para practicar os proponemos que intentéis deducir vosotros la evolución en la salida de un contador cíclico de 3 bits binario natural con las señales de entrada adicionales que acabamos de ver suponiendo los siguientes cambios:

- La entrada externa *E* está puesta a 101.
- La entrada $C/\bar{L}$ funciona de manera asíncrona.

Posteriormente podéis mirar de deducir la misma evolución en el supuesto que la entrada de *Reset* fuese asíncrona.

8.2. Diseño de contadores síncronos

Hemos visto que la estructura elemental de un contador síncrono es un registro EP/SP, es decir, tantos biestables como bits tenga el código de contaje activados todos ellos por un mismo sincronismo externo, más un sistema combinacional que calcula el siguiente valor de salida a partir del valor actual del código de contaje.

8.2.1. Implementación del sistema combinacional

El diseño de un contador por consiguiente se reduce básicamente a diseñar este sistema combinacional. Esto se puede resolver con una tabla de verdad en la que como entradas tenemos la salida del contador (valor actual) y como salidas se especifica el siguiente valor de contaje para cada posible entrada.

Posteriormente veremos cómo se tiene en consideración la presencia de señales de entrada adicionales a la entrada de sincronismo elemental.

Ejemplo 8.2

Vamos a diseñar por ejemplo un contador cíclico binario natural de 3 bits.

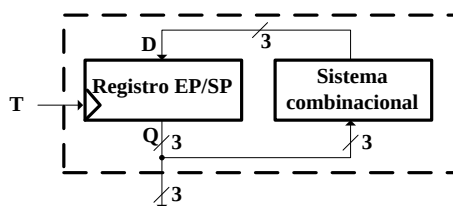


Figura 8.4. Estructura elemental de un contador de 3 bits.

Para ello diseñaremos un sistema combinacional que cumpla la siguiente tabla de verdad.

Q_2	Q_1	Q_0	D_2	D_1	D_0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

Tabla 8.1. Tabla de verdad del sistema combinacional correspondiente a un contador cíclico binario natural de 3 bits.

Si planteamos por ejemplo tres tablas de Karnaugh para diseñar los sistemas correspondientes a D_2 , D_1 y D_0 y simplificamos dichas funciones por uno entonces obtendremos que $D_2 = \overline{Q_0}Q_2 + \overline{Q_1}Q_2 + \overline{Q_2}Q_1Q_0$, $D_1 = \overline{Q_0}Q_1 + \overline{Q_1}Q_0 = Q_1 \oplus Q_0$ y $D_0 = \overline{Q_0}$.

De este modo el contador se limita a implementar el siguiente sistema.

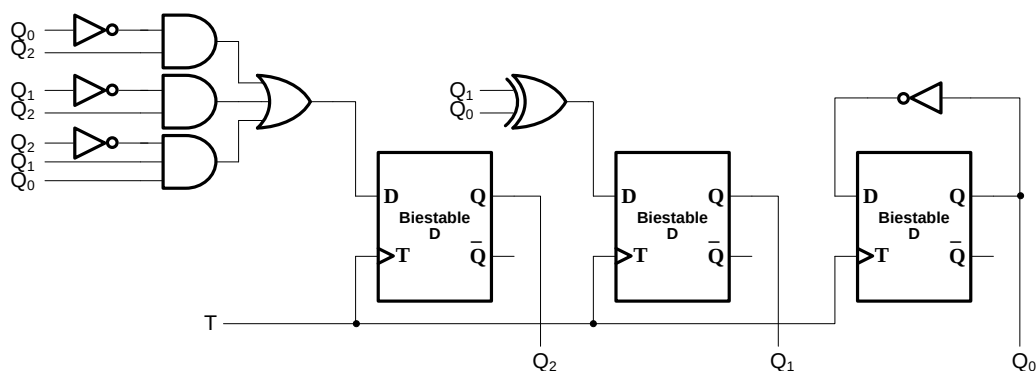


Figura 8.5. Estructura elemental de un contador de 3 bits. Vemos que está formado por tres biestables que comparten un sincronismo externo T . La entrada de cada biestable procede de un sistema combinacional que, a partir de los bits de salida de los biestables, es decir, a partir de la salida del contador, determina cuáles han de ser los próximos valores a los que tienen que cargarse los biestables según la tabla de verdad que hemos visto.

Si el contador es cíclico hemos visto que la salida para la última combinación de la tabla de verdad es la primera del código de contaje. Si el contador hubiese sido anclado entonces la salida para esta última combinación sería la misma última combinación. Si el contador hubiese sido no anclado entonces la salida para esta última combinación hubiera sido indeterminada. Estas diferencias habrían dado lugar a diferencias en los sistemas combinacionales que tenemos que aplicar a las entradas de los biestables.

Q_2	Q_1	Q_0	D_2	D_1	D_0	Q_2	Q_1	Q_0	D_2	D_1	D_0	Q_2	Q_1	Q_0	D_2	D_1	D_0
0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	1
0	0	1	0	1	0	0	0	1	0	1	0	0	0	1	0	1	0
0	1	0	0	1	1	0	1	0	0	1	1	0	1	0	0	1	1
0	1	1	1	0	0	0	1	1	1	0	0	0	1	1	1	0	0
1	0	0	1	0	1	1	0	0	1	0	1	1	0	0	1	0	1
1	0	1	1	1	0	1	0	1	1	1	0	1	0	1	1	1	0
1	1	0	1	1	1	1	1	0	1	1	1	1	1	0	1	1	1
1	1	1	0	0	0	1	1	1	1	1	1	1	1	1	X	X	X

Tabla 8.2. Diferencias entre las tablas de verdad del sistema combinacional correspondiente a un contador binario natural de 3 bits en el caso de ser cíclico (izquierda), anclado (central) y no anclado (derecha).

Este procedimiento que acabamos de ver se generaliza para el diseño de cualquier contador. Tan solo tenemos que conocer el código de contaje para poder plantear la tabla de verdad de la que se obtienen los sistemas combinacionales que se aplican a las entradas de los biestables.

8.2.2. Implementación de la señal $U/\bar{D}$

En el caso que el contador presente una entrada de tipo $U/\bar{D}$ ésta se considera como una entrada más de la tabla de verdad, puesto que según su valor entonces la salida a la que tiene que tender el contador es el siguiente valor del código de contaje o bien el anterior.

Ejemplo 8.3

Vamos a diseñar, por ejemplo, un contador anclado de código Gray de 2 bits con entrada adicional de tipo $U/\bar{D}$. Básicamente el problema se reduce a diseñar un sistema combinacional que responda a la siguiente tabla de verdad que aplicaremos a las entradas de los biestables que configuran el registro donde se almacena el valor actual.

$U/\bar{D}$	Q_1	Q_0	D_1	D_0
0	0	0	0	0
0	0	1	0	0
0	1	1	0	1
0	1	0	1	1
1	0	0	0	1
1	0	1	1	1
1	1	1	1	0
1	1	0	1	0

Tabla 8.3. Tabla de verdad del sistema combinacional correspondiente a un contador anclado en código Gray de 2 bits con entrada adicional de tipo $U/\bar{D}$. Vemos que al ser anclado si el contador está en la última combinación y ha de contar en modo ascendente entonces su valor no varía. Así mismo, si el contador está en la primera combinación y ha de contar en modo descendente, tampoco varía.

En este ejemplo, si planteamos las tablas de Karnaugh correspondientes a D_1 y D_0 y las simplificamos por unos obtendremos como resultado que $D_1 = Q_1 Q_0 + Q_0 U/\bar{D} + Q_1 U/\bar{D}$ y que $D_0 = \bar{Q}_1 U/\bar{D} + Q_1 \bar{U}/\bar{D}$.

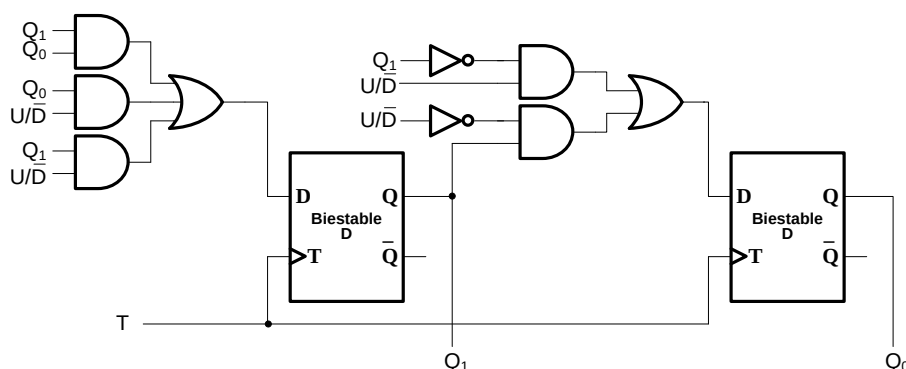


Figura 8.6. Estructura elemental de un contador anclado de 2 bits con código Gray y señal de entrada de tipo $U/\bar{D}$.

Ejemplo 8.4

Os proponemos que diseñéis un contador no anclado de 3 bits con entrada adicional de tipo $U/\bar{D}$ que siga el siguiente código de conteo que nos hemos inventado: 000, 010, 111 y 101.

En este punto sólo nos queda por ver cómo se diseñan contadores con señales adicionales de control del tipo *Enable*, *Reset* o $C/\bar{L}$, pudiendo algunas de ellas tener un comportamiento síncrono o bien asíncrono.

Anteriormente hemos visto que el diseño de un contador elemental, tanto si presenta señal de entrada $U/\bar{D}$ como si no, se limita a diseñar un cierto sistema combinacional. En el caso de tener que implementar señales adicionales de control éstas no afectan al sistema combinacional sino que son entradas adicionales que actúan sobre los biestables en los que se almacena el valor actual del contador. Si estas señales adicionales de control han de actuar sobre el contador de forma asíncrona, los

biestables tendrán que disponer de entradas de tipo *Set* o *Reset*. Pasemos a ver distintos casos.

8.2.3. Implementación de la señal de *Reset*

La señal de *Reset* hace que el contador se actualice al primer valor del código de contaje. Esta señal puede tener un comportamiento síncrono o asíncrono.

En el primer caso la actualización se realiza en el momento que llega un flanco de sincronismo, por lo que su implementación se hará mediante la entrada *D* en cada uno de los biestables que configuran el registro del contador, tal y como muestra la siguiente figura.

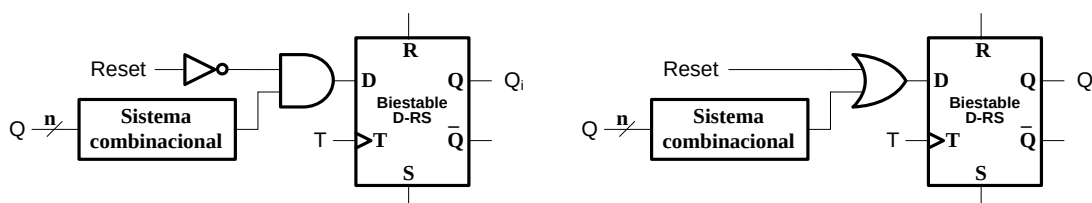


Figura 8.7. Implementación de la señal de *Reset* con funcionamiento síncrono mediante puertas lógicas. Podemos observar que cuando llega un sincronismo el biestable actualiza su salida según el sistema combinacional propio del contador o bien actualiza su salida a cero (esquema de la izquierda) o a uno (esquema de la derecha), dependiendo el valor correspondiente a dicho bit en la primera combinación del código de contaje. Recordemos que este esquema se repite para cada uno de los biestables que contenga el contador.

Otra solución a una implementación de la señal de *Reset* con funcionamiento síncrono es mediante multiplexores en la entrada de los biestables.

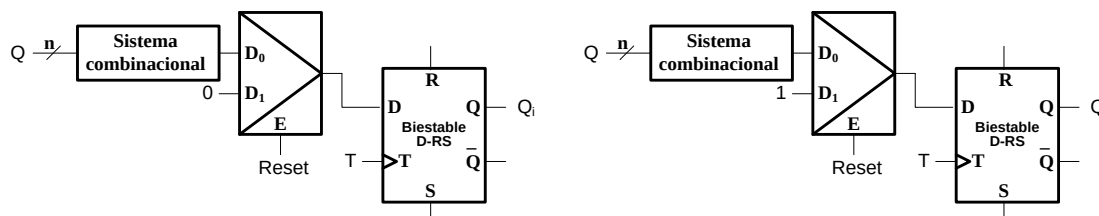


Figura 8.8. Implementación de la señal de *Reset* con funcionamiento síncrono mediante multiplexores. En este caso según el valor de *Reset* el multiplexor envía a la entrada del biestable el valor del sistema combinacional o bien un cero (esquema de la izquierda) o un uno (esquema de la derecha), dependiendo el valor correspondiente a dicho bit en la primera combinación del código de contaje.

La señal de *Reset* también puede funcionar de manera asíncrona. En este caso el contador se actualiza al primer valor del código de contaje tan pronto como se activa *Reset*. Para ello su implementación no puede realizarse por medio de las entradas síncronas de los biestables sino por medio de sus entradas asíncronas, es decir, las entradas de *Set* o *Reset* propias del biestable, según si el biestable ha de ponerse a uno o bien a cero, respectivamente.

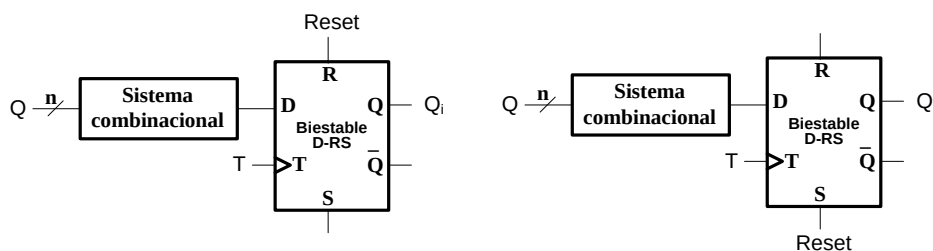


Figura 8.9. Implementación de la señal de *Reset* con funcionamiento asíncrono. Según el valor de *Reset* el biestable se actualiza inmediatamente a cero (esquema de la izquierda) o a uno (esquema de la derecha), dependiendo el valor correspondiente a dicho bit en la primera combinación del código de conteo.

En los esquemas vistos la señal de *Reset* funciona por lógica positiva. Se podría plantear una señal de *Reset* que funcionase por lógica negativa y las soluciones vistas continúan siendo válidas más allá del hecho que tendríamos que invertir dicha señal.

Ejemplo 8.5

Vamos a implementar el esquema de un contador de 3 bits no anclado con *Reset* síncrono cuyo código de conteo es el siguiente 010, 011, 101 y 110.

Por un lado, diseñamos el sistema combinacional que responderá a la siguiente tabla de verdad.

Q_2	Q_1	Q_0	D_2	D_1	D_0
0	1	0	0	1	1
0	1	1	1	0	1
1	0	1	1	1	0
1	1	0	X	X	X
resto			X	X	X

Tabla 8.4. Tabla de verdad del sistema combinacional de un contador no anclado cuyo código de conteo es 010, 011, 101 y 110.

Si simplificamos por el método de ceros de Karnaugh las funciones D_2 , D_1 y D_0 obtendremos $D_2 = Q_2 + \bar{Q}_1 + Q_0$, $D_1 = Q_2 + \bar{Q}_1 + \bar{Q}_0$ y $D_0 = \bar{Q}_2 + Q_1 + \bar{Q}_0$. Estos tres sistemas combinacionales se tendrían que aplicar a la entrada de 3 biestables para generar, a cada flanco de sincronismo, el siguiente valor del código de conteo a partir del valor actual que éste presente.

Nos queda finalmente plantear cómo implementar la señal de *Reset*. Al tratarse de una señal de funcionamiento síncrono podemos optar por cualquiera de las dos soluciones comentadas anteriormente, eligiendo por ejemplo la primera que hemos visto con puertas lógicas. Hemos de tener presente que como que la primera combinación del código de conteo es 010 entonces el biestable que da la salida de mayor peso ha de ponerse a cero cuando se active *Reset*, el biestable de peso intermedio ha de ponerse a uno y el de menor peso ha de ponerse a cero.

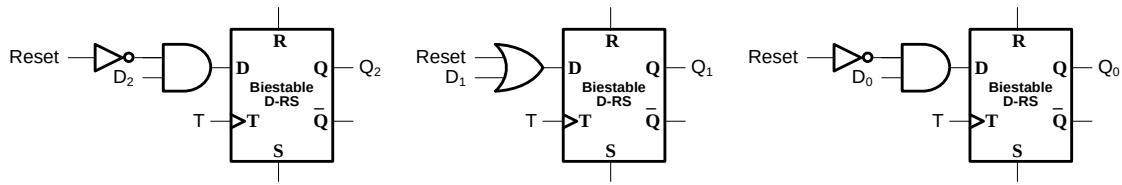


Figura 8.10. Implementación de un contador de 3 bits no anclado con código de conteo 010, 011, 101 y 110. Se incluye una señal de *Reset* con funcionamiento síncrono. La implementación de los tres sistemas combinacionales no se ha esquematizado y simplemente se muestra dónde se aplicaría cada uno de ellos (D_2 , D_1 , D_0).

En algunos contadores, poco habituales, podemos hallar una entrada de tipo *Set* en vez de *Reset*. Cuando se activa, el contador se posiciona en la última combinación del código de conteo en vez de en la primera, por lo que las soluciones son las mismas que acabamos de ver sólo que tendremos que observar cuál es la última de las combinaciones del código de conteo para saber el valor al que han de actualizarse los biestables.

8.2.4. Implementación de la señal de $C/\bar{L}$

La señal de $C/\bar{L}$ permite o bien que el contador funcione de manera normal si $C/\bar{L} = 1$, es decir, contando cada vez que recibe un flanco de sincronismo, o bien actualizándose a un valor dado por una entrada externa cuando $C/\bar{L} = 0$, a partir del cual proseguirá su conteo según su código.

Como en el caso de la señal de *Reset*, el funcionamiento de esta señal $C/\bar{L}$ para la carga externa puede ser de tipo síncrono o de tipo asíncrono, es decir, cuando llegue un flanco de sincronismo o bien en el momento de su activación.

En el caso del funcionamiento síncrono hemos de actuar sobre la entrada *D* de cada biestable. Para ello se suele poner un multiplexor en la entrada del biestable para que envíe el valor correspondiente al sistema combinacional propio del contador o bien el valor de una entrada externa E_i , según el valor de la señal $C/\bar{L}$.

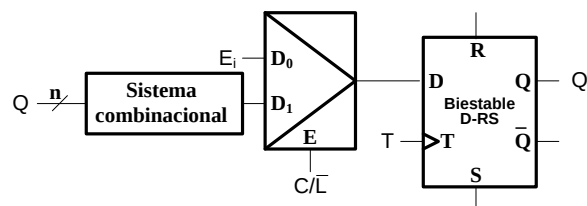


Figura 8.11. Implementación de la señal de $C/\bar{L}$ con funcionamiento síncrono mediante multiplexores. En este caso según el valor de $C/\bar{L}$ el multiplexor envía a la entrada del biestable el valor del sistema combinacional o bien el valor de una entrada externa E_i .

Para el caso del funcionamiento asíncrono actuaremos sobre las entradas asíncronas del biestable, diseñando un simple sistema con puertas lógicas para que cuando $C/\bar{L} = 0$ entonces el biestable se ponga a cero si la entrada externa $E_i = 0$, actuando en este caso sobre la entrada *Reset* del biestable, o bien que se ponga a uno si la entrada externa $E_i = 1$, actuando en este caso sobre la entrada *Set* del biestable.

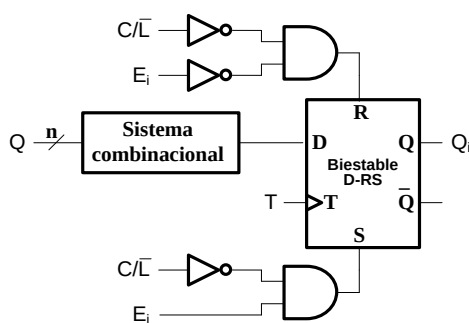


Figura 8.12. Implementación de la señal de $C/\bar{L}$ con funcionamiento asíncrono. Vemos que cuando $C/\bar{L} = 0$ entonces el biestable se pone a cero o uno, según E_i , puesto que siempre activaremos una de sus dos entradas asíncronas.

En estos esquemas hemos supuesto que cuando esta señal vale uno entonces el contador va contando de manera síncrona mientras que cuando vale cero entonces el contador realiza una carga, síncrona o asíncrona. Lógicamente, el funcionamiento podría haber sido inverso, es decir, que cuando esta señal vale uno entonces se realiza la carga mientras que cuando vale cero se realiza el conteo, indicándose entonces habitualmente como $\bar{C}/L$. En este caso las posibles implementaciones continúan siendo las mismas pero intercambiando las entradas del multiplexor para la solución síncrona o no poniendo los inversores en la señal $\bar{C}/L$ para la solución asíncrona.

Ejemplo 8.6

Vamos a implementar el esquema de un contador binario natural de 2 bits anclado con una posible carga externa mediante una señal $\bar{C}/L$ de tipo asíncrono.

Como es habitual primero tendríamos que diseñar el sistema combinacional del contador, en este caso respondiendo a la siguiente tabla de verdad.

Q_1	Q_0	D_1	D_0
0	0	0	1
0	1	1	0
1	0	1	1
1	1	1	1

Tabla 8.5. Tabla de verdad del sistema combinacional de un contador binario natural de 2 bits anclado.

Si simplificamos por el método de ceros de Karnaugh las funciones D_1 y D_0 obtendremos $D_1 = Q_1 + Q_0$ y $D_0 = Q_1 + \bar{Q}_0$.

Posteriormente hemos de aplicar a cada biestable la solución que acabamos de comentar para realizar una posible carga externa asíncrona cuando $\bar{C}/L = 1$ a partir de un bus externo de 2 bits, E_1 y E_0 .

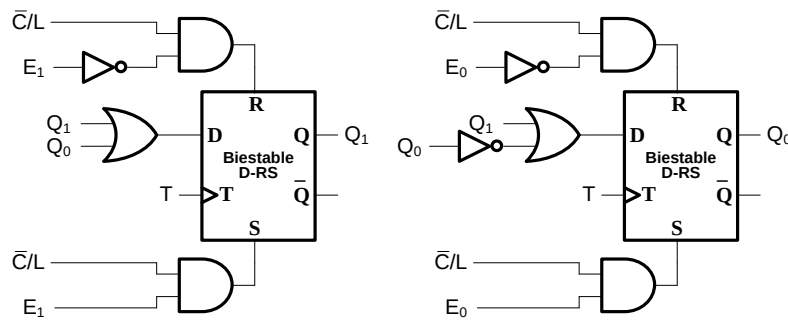


Figura 8.13. Implementación de un contador binario natural de 2 bits anclado con entrada adicional de carga externa asíncrona.

8.2.5. Implementación de la señal de *Enable*

La señal *Enable* habilita la entrada de sincronismo del contador de tal manera que cuando llega un pulso entonces el contador pasa al siguiente valor del código de contaje. En caso de no estar habilitado no variará su salida por mucho que lleguen sincronismos. Para esta señal no procede diferenciar un funcionamiento asíncrono puesto que lo que habilita es el flanco de sincronismo.

Su implementación elemental pasa por aplicar un multiplexor a la entrada *D* de cada uno de los biestables del contador para que éste envíe al biestable el valor del sistema combinacional cuando está habilitado o bien para que envíe al biestable el mismo valor que ya tenía almacenado cuando está deshabilitado.

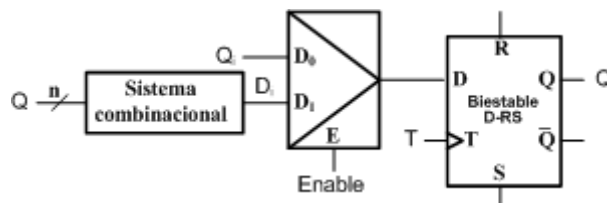


Figura 8.14. Implementación de la señal de *Enable* de un contador. Vemos que el multiplexor hará que el biestable se actualice según el mismo valor que ya tenía si *Enable* = 0 o bien al valor que le corresponda si *Enable* = 1 para avanzar en el contaje.

Con esta solución lo que se pretende es que, sin tener que tocar la señal de sincronismo, el biestable actualice su salida al nuevo valor que le corresponda si *Enable* = 1 o bien al mismo valor que tenía si *Enable* = 0, de tal modo que en este último caso al usuario le daría la sensación como si el biestable no hubiera atendido la petición de sincronismo *T*. El motivo es que en los sistemas digitales que funcionan con sincronismos existe un principio por el que se recomienda que la señal de sincronismo “no se modifique” y llegue por igual a todos los elementos que la usen, con la finalidad de no introducir retardos en esta línea y que unos elementos atiendan las peticiones antes que los otros.

8.2.6. Establecimiento de prioridades

Un contador suele tener varias señales de entrada como las que hemos visto en los apartados anteriores, por lo que su implementación puede incorporar varias de las soluciones planteadas.

Ante este hecho puede darse el caso que varias de estas entradas se activen simultáneamente pero que en cambio sean contradictorias. Por ejemplo ¿qué sucede si se activa la señal de *Reset* y simultáneamente se desactiva la señal de *Enable*? O bien ¿qué sucede si se activan simultáneamente una señal de *Reset* y una de *Set*?

La solución no es universal y de hecho siempre ha de especificarse cuál es la prioridad que hemos de considerar ante estas posibles activaciones que entran en conflicto. Aún no existiendo una solución universal sí que se puede tomar en consideración la idea que la máxima prioridad la tendremos en los elementos más cercanos al biestable puesto que estos son los “últimos” en afectar al biestable. Veámoslo con algún ejemplo.

Ejemplo 8.7

Diseñar un contador cíclico cuyo código de conteo se corresponde a la evolución de la secuencia 00, 01 y 10, que tendrá:

- Una entrada *Enable* que funciona por lógica negativa y es prioritaria respecto cualquier otra señal.
- Una entrada *Reset* que funciona por lógica positiva y síncrona.
- Una entrada *Set* que funciona por lógica positiva y síncrona y que es más prioritaria que *Reset*.

Como en todos los contadores, primero tendríamos que diseñar el sistema combinacional que nos daría las entradas de los dos biestables que constituyen su registro. Para centrarnos en las prioridades dejamos para vosotros que realicéis el diseño de dichos sistemas, siendo la solución $D_1 = \overline{Q_1} \times Q_0$ y $D_0 = \overline{Q_1} \times \overline{Q_0}$.

Si ahora nos centramos en la implementación simultánea de las señales *Enable*, *Reset* y *Set*, vemos que las tres soluciones pasan por poner multiplexores en las entradas *D* de cada uno de los dos biestables que tendremos, puesto que todas ellas funcionan de manera síncrona. Como que la señal *Reset* es la menos prioritaria, seguida de *Set* y finalmente *Enable*, éste será el orden de los multiplexores, es decir el más lejano al biestable será el que dependa de *Reset* y el más cercano el que dependa de *Enable*.

Recordemos que cuando se activa el *Reset* los biestables han de adoptar la primera combinación del código (en este caso 00), para el *Set* han de adoptar la última (en este caso 10) y para *Enable* = 1 (deshabilitado puesto funciona por lógica negativa según enunciado) han mantener el mismo valor que ya tenían.

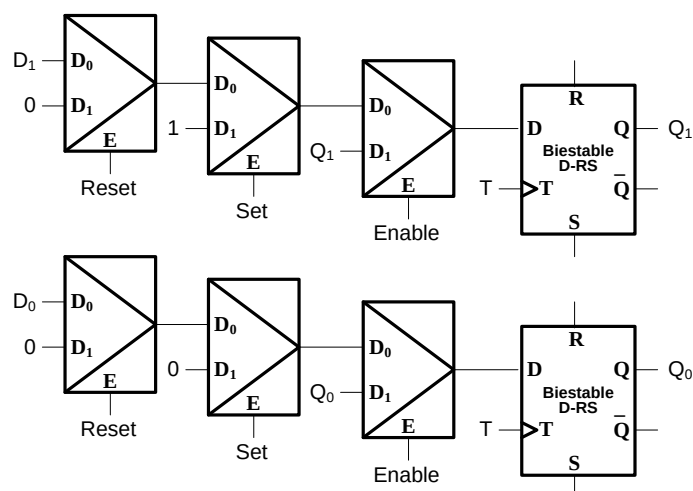


Figura 8.15. Implementación de un contador cíclico con código de conteo 00, 01 y 10 con entradas *Enable* (por lógica negativa), *Set* y *Reset* síncronas y siendo ésta la prioridad, de mayor a menor. Vemos que cuando se activa *Reset* el primer multiplexor tiende a enviar a los biestables la primera combinación del código (00), si se activa *Set* el segundo multiplexor tiende a enviar la última combinación del código (10) y si no es activa *Enable*, es decir *Enable* = 1 al ser por lógica negativa, el último multiplexor tiende a enviar a los biestables el mismo valor que ya tenían. Observamos que este último multiplexor es más prioritario al ser el más cercano al biestable.

No todas las entradas han de funcionar de manera síncrona o bien todas de manera asíncrona. Así, si por ejemplo tomamos este último ejemplo y consideramos que la entrada *Set* fuera asíncrona, entonces la solución pasaría por mantener los otros dos multiplexores que implementan *Enable* y *Reset*, pero tendremos que diseñar una solución asíncrona para *Set*, es decir, actuando sobre las entradas asíncronas de los biestables, que además ha de respetar la prioridad indicada.

Como que para *Set* el contador ha de ponerse a 10, el biestable que da Q_1 habrá de ponerse a uno y actuaremos sobre su entrada asíncrona de *Set*, mientras que el otro biestable Q_0 habrá de ponerse a cero y actuaremos sobre la entrada asíncrona de *Reset* de este segundo biestable. Además, y por el hecho que *Enable* es más prioritario que *Set*, estas entradas asíncronas de los biestables que atienden la petición de *Set* sólo habrán de activarse si *Enable* = 0 (recordemos que funciona por lógica negativa).

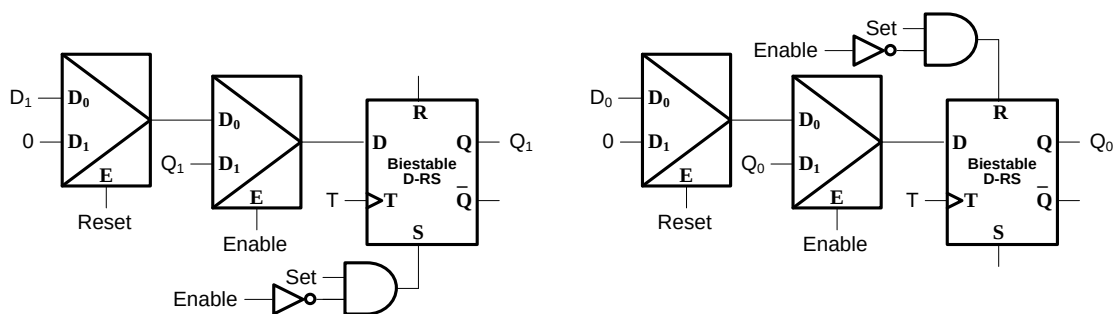


Figura 8.16. Implementación de un contador cíclico con código de conteo 00, 01 y 10 con entradas *Enable* (por lógica negativa), *Set* asíncrono y *Reset* síncrono y siendo ésta la prioridad, de mayor a menor.

Ejemplo 8.8

Para practicar os proponemos que intentéis diseñar un contador con las siguientes características:

- Código de conteo binario natural de 3 bits de 0 a 5, es decir 000, 001, 010, 011, 100 y 101.
- Funcionamiento cíclico.
- Señal de *Enable* que funciona por lógica positiva.
- Señal de *Set* que funciona por lógica negativa y de manera síncrona.
- Señal de *Reset* que funciona por lógica negativa y de manera asíncrona.
- Señal de conteo o carga externa $C/\bar{L}$ que funciona de manera síncrona.
- En caso de activarse más de una entrada simultáneamente la prioridad viene dada por *Enable*, *Set*, *Reset* y $C/\bar{L}$, siendo *Enable* la más prioritaria y $C/\bar{L}$ la menos prioritaria.

8.3. Contadores con combinaciones repetidas

Hasta el momento todos los contadores que hemos visto presentan, dentro de su código de conteo, combinaciones diferentes.

Pero en algunos casos nos pueden requerir el diseño de un contador en el que alguna o algunas combinaciones se repitan. Así por ejemplo, ¿cómo se diseña un contador cíclico cuyo código de conteo es 00, 01, 10, 00 y 11? La solución no es trivial puesto que cuando diseñamos la tabla de verdad nos daremos cuenta que cuando el contador presenta la combinación 00 no somos capaces de determinar si la siguiente combinación es la 01 o bien la 11.

Q_1	Q_0	D_1	D_0
0	0	0	1
0	1	1	0
1	0	0	0
0	0	1	1
1	1	0	0

Tabla 8.6. Aparente tabla de verdad del sistema combinacional de un contador que sigue la secuencia 00, 01, 10, 00 y 11. Vemos que la tabla no es coherente puesto que en el margen izquierdo en el que se especifican las combinaciones de entrada del sistema combinacional aparece repetida la combinación 00 y esto no es posible en un sistema combinacional en el que por definición para cada posible entrada hemos de tener una única salida. Otra incoherencia está en la última fila de la tabla donde se especifica que después de la combinación 11 hemos de volver a la 00 pero ésta, ¿cómo sabemos que es la primera?

En el caso de tener combinaciones repetidas en un código de conteo la solución pasa por poder diferenciar estas combinaciones y la única manera de hacerlo, teniendo en cuenta que forman parte de un sistema combinacional, es añadir bits al código para que con ellos se puedan diferenciar todas las combinaciones.

Ejemplo 8.9

Para entenderlo volvamos al diseño de un contador que ha de seguir de manera cíclica la secuencia 00, 01, 10, 00 y 11.

En este caso será necesaria la inclusión de un bit en el sistema combinacional que permita diferenciar las combinaciones repetidas. A este nuevo bit de hecho sólo será necesario darle valores concretos para las combinaciones repetidas a fin de diferenciarlas, pudiendo tomar cualquier otro valor para el resto de combinaciones.

Q_2	Q_1	Q_0	D_2	D_1	D_0
0	0	0	X	0	1
X	0	1	X	1	0
X	1	0	1	0	0
1	0	0	X	1	1
X	1	1	0	0	0

Tabla 8.7. Tabla de verdad del sistema combinacional correspondiente a un contador cíclico cuyo código de conteo es 00, 01, 10, 00 y 11. Al tener repetida la combinación 00 necesitamos definir un nuevo bit que permita diferenciarlas, asignándole en un caso el valor 0 y en el otro el valor 1. De esta manera el sistema combinacional pasa a ser unívoco y para cada combinación de entrada queda clara la única posible combinación de salida. Vemos también que para las combinaciones no repetidas el valor que adopte este nuevo bit es indiferente puesto que el resto de bits claramente marcarán el valor actual de conteo.

De esta manera el proceso de diseño continúa siendo válido, simplemente teniendo presente que el sistema combinacional no es de sólo 2 bits sino de 3. Un posible sistema que cumple esta tabla sería $D_2 = \overline{Q_1} + \overline{Q_0}$, $D_1 = \overline{Q_1} \times Q_0 + Q_2 \times \overline{Q_1} \times \overline{Q_0}$ y $D_0 = \overline{Q_1} \times \overline{Q_0}$.

Cabe remarcar que este nuevo bit dará lugar a un nuevo biestable, es decir, nuestro contador dispondrá de 3 biestables para almacenar el valor actual del código de conteo, si bien sólo 2 de ellos serán los que propiamente indicarán los bits de salida del contador. Dicho de otra manera, el bus de salida del contador continúa siendo de sólo 2 bits si bien internamente dispone de un tercero.

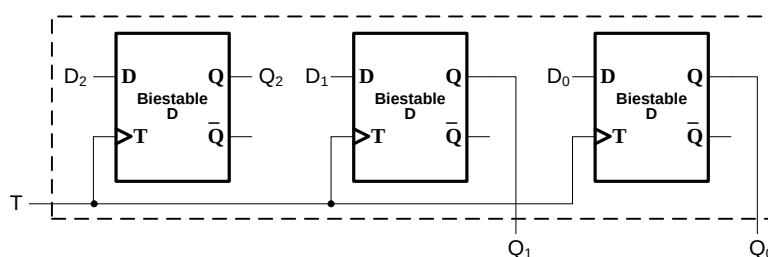


Figura 8.17. Implementación de un contador de 2 bits con combinaciones repetidas. Vemos que ha sido necesaria la inclusión de un tercer biestable que a partir del sistema combinacional del contador permitirá diferenciar “internamente” las combinaciones repetidas. Lógicamente las entradas de los biestables dependen de los tres valores Q_2 , Q_1 y Q_0 ; pero no obstante las salidas del contador desde el punto de vista del usuario final sólo son Q_1 y Q_0 .

A partir del ejemplo anterior podemos extrapolar que en algunos casos puede ser necesaria la inclusión de más de un bit (y biestable) al contador en el supuesto que tengamos una combinación con múltiples repeticiones.

Ejemplo 8.10

Supongamos que tenemos que diseñar un contador anclado que siga la siguiente secuencia 00, 01, 00, 11, 01, 00.

En este caso vemos que tenemos 3 veces la combinación 00 lo que nos fuerza a añadir 2 bits al sistema combinacional que determina la secuencia de conteo puesto que con un único bit no podemos diferenciar 3 valores repetidos.

Por otro lado, vemos que también tenemos repetida la combinación 01. En este caso sería necesario un bit más, aunque lógicamente no hace falta que sea un bit más “adicional” a los otros 2 bits que ya hemos añadido para la repetición de la combinación 00 puesto que podemos aprovechar cualquiera de ellos.

En este caso pues una posible definición del sistema combinacional del contador que estamos planteando podría ser la siguiente.

Q_3	Q_2	Q_1	Q_0	D_3	D_2	D_1	D_0
0	0	0	0	X	0	0	1
X	0	0	1	0	1	0	0
0	1	0	0	X	X	1	1
X	X	1	1	X	1	0	1
X	1	0	1	1	X	0	0
1	X	0	0	1	X	0	0
resto				X	X	X	X

Tabla 8.8. Tabla de verdad del sistema combinacional correspondiente a un contador anclado cuyo código de conteo es 00, 01, 00, 11, 01 y 00. Al tener repetida 3 veces la combinación 00 necesitamos definir un par de bits que permitan diferenciarlas. Además, uno de estos bits también lo usaremos para diferenciar las combinaciones repetidas 01.

A partir de la tabla de verdad implementaríamos las cuatro funciones D_3 , D_2 , D_1 y D_0 que aplicaríamos a los cuatro biestables del registro que configura el contador, si bien sólo un par de ellos propiamente darían las salidas externas que muestran la evolución del código de conteo.

Es importante apreciar que el hecho de tener combinaciones repetidas sólo afecta al diseño del sistema combinacional del contador. No obstante en estos contadores con combinaciones repetidas podemos tener señales adicionales de control como las vistas en apartados anteriores de tipo *Enable*, *Reset*, $C/\bar{L}$, etc. que condicionarán el hecho de tener que incluir las soluciones necesarias para su implementación, tal y como hemos visto en apartados anteriores.

Ejemplo 8.11

Imaginemos que nos piden diseñar un contador con las siguientes características:

- Código de conteo en binario correspondiente a la secuencia decimal 1, 2, 0, 3, 2, 0 y 1.
- Comportamiento cíclico.
- Entrada *Enable* por lógica positiva y prioritaria respecto cualquier otra.
- Entrada *Reset* por lógica negativa y asíncrona.

- Entrada $C/\bar{L}$ para efectuar una posible carga síncrona de un bus externo E cuando $C/\bar{L} = 0$, siendo menos prioritaria que la entrada *Reset*.

En este caso y como en todos los contadores tenemos que diseñar por un lado el sistema combinacional y por otro lado plantear la estructura de los biestables que pueda satisfacer los requerimientos de las señales de control adicionales.

Para el sistema combinacional vemos que tenemos repetidas las combinaciones binarias correspondientes a los valores 0, 1 y 2. Como que cada una de ellas sólo está repetida una vez, con un único bit adicional tendremos suficiente. Por consiguiente, una posible tabla de verdad de este sistema combinacional podría ser la siguiente.

Q_2	Q_1	Q_0	D_2	D_1	D_0
0	0	1	0	1	0
0	1	0	0	0	0
0	0	0	X	1	1
X	1	1	1	1	0
1	1	0	1	0	0
1	0	0	1	0	1
1	0	1	0	0	1
resto			X	X	X

Tabla 8.9. Tabla de verdad del sistema combinacional correspondiente a un contador cíclico cuyo código de conteo es 01, 10, 00, 11, 10, 00 y 01. Al tener combinaciones repetidas pero cada una de ellas sólo 2 veces, con un único bit adicional tendremos suficiente para diferenciar todas las combinaciones.

A partir de la tabla de verdad diseñaríamos el sistema combinacional correspondiente a D_2 , D_1 y D_0 .

Llegado este punto sólo quedaría plantear la estructura de los biestables que cumplan los requerimientos de las entradas de control. Por un lado tenemos la señal *Enable* que al ser prioritaria respecto al resto la implementaremos mediante un multiplexor puesto justo a la entrada de cada biestable. Por otro lado tenemos la señal *Reset* que al ser asíncrona aplicaremos a las entradas asíncronas de cada biestable con la finalidad que cuando se active ponga inmediatamente los biestables a 001, al ser ésta la primera combinación del código de conteo. Cabe mencionar que al indicarnos que *Enable* es más prioritario tendremos que añadir las puertas lógicas que sean necesarias a estas entradas asíncronas para validar que sólo puedan activarse si, además de tener una consigna de *Reset*, también tenemos que $Enable = 1$. Finalmente la señal de $C/\bar{L}$ síncrona que al ser menos prioritaria se implementará con otro multiplexor puesto antes del multiplexor de *Enable*.

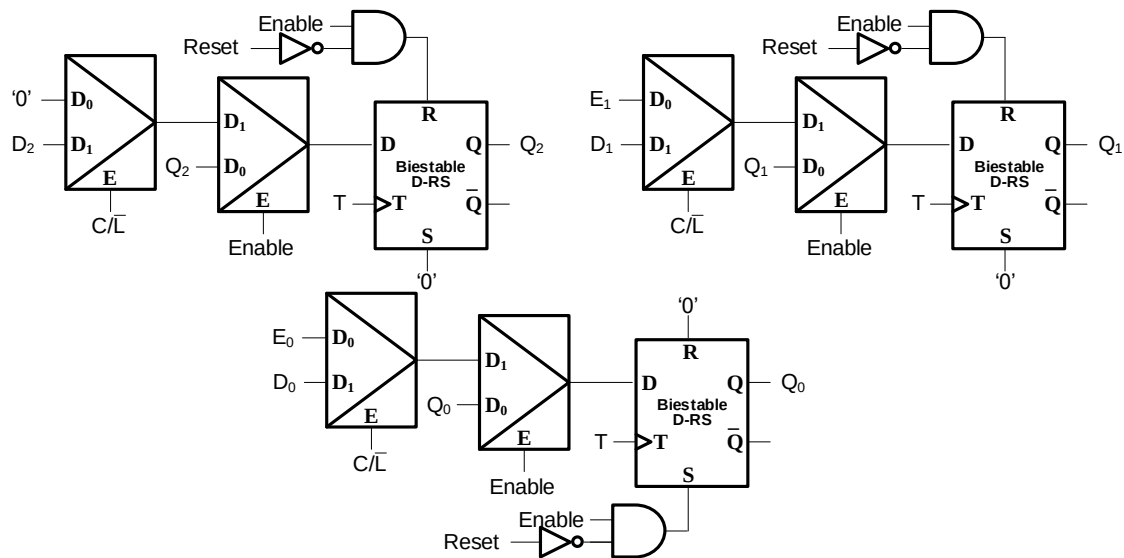


Figura 8.18. Implementación de un contador cíclico cuyo código de conteo es 01, 10, 00, 11, 10, 00 y 01, en el que al tener combinaciones repetidas se ha añadido un tercer biestable. Vemos que dispone de entrada *Enable* prioritaria respecto cualquier otra señal. También hay una señal de *Reset* que es activa por lógica negativa y que, en caso de tener también *Enable* = 1, forzará que los biestables cambien asincrónicamente a la combinación 001. Finalmente vemos una entrada *C/L* que permite una carga externa siempre que, además de activarse, tengamos *Enable* = 1 y *Reset* = 1. Recordemos que las entradas asíncronas de un biestable son prioritarias respecto las síncronas y por eso no hace falta ningún control adicional de priorización en el multiplexor que depende de *C/L*. Es interesante observar que el multiplexor de *C/L* correspondiente al biestable de mayor peso no tiene una entrada externa asociada sino que está puesta a cero. Esto es normal puesto que, al tratarse de un contador de 2 bits, del mismo modo que sólo hay 2 bits de salida sólo tiene sentido que existan 2 posibles bits de entrada externos. Con esta solución deducimos que cuando se cargue externamente una posible combinación que tenga una repetición iremos siempre a la primera de las combinaciones repetidas, habiendo sido una decisión de diseño nuestra.

Ejemplo 8.12

Para practicar os proponemos que diseñéis un contador con las siguientes características:

- Código de conteo binario correspondiente a la secuencia decimal 0, 1, 0, 2, 0, 3, 0.
- Al llegar a la última combinación del código queda anclado.
- Señal de *Reset* asíncrona y activa por lógica negativa.
- Señal de $\bar{C}/L$ que cuando vale 0 permite una carga en paralelo asíncrona. En el supuesto de cargar una combinación repetida siempre irá a la primera de ellas.
- La señal $\bar{C}/L$ es más prioritaria que la señal *Reset*.

8.4. Ampliación de la capacidad de conteo

En los apartados anteriores hemos visto cómo se diseña un contador. Desde el punto de vista de contadores comerciales el abanico de opciones no es tan amplio y la mayoría son contadores que cuentan siguiendo códigos binarios habituales. La capacidad de conteo de estos contadores depende básicamente del código que se use y del número

de bits del correspondiente código. Así, por ejemplo, con un código binario de 3 bits sólo se puede contar hasta 7 o con un código BCD de 4 bits sólo hasta el 9.

Si disponemos de uno de estos contadores comerciales y queremos aumentar su capacidad de conteo la solución habitual pasa por encadenar contadores. Un ejemplo muy claro lo tenemos con los contadores BCD de 4 bits puesto que con un contador podemos llegar hasta el 9 pero con dos contadores que pudieran encadenarse podríamos contar hasta el 99, uno para las decenas y otro para las unidades, pudiendo entonces, por ejemplo, implementar un marcador electrónico de una competición deportiva.

Existen dos maneras de ampliar la capacidad de conteo basadas en el encadenamiento de contadores: asincrónamente o síncronamente, en función de cómo se realiza la comunicación entre los distintos contadores encadenados.

8.4.1. Encadenamiento de contadores de forma asíncrona

En este caso el sincronismo que usan los contadores es distinto. El primero de ellos recibe el sincronismo externo habitual T mientras que el segundo recibe un sincronismo que proviene del primer contador. De esta manera cuando el primer contador ha llegado al límite de su capacidad de conteo y ha de reiniciarse entonces “avisa” al segundo contador para que éste tenga presente que ha de incrementarse en uno. Para ello es necesario que el contador disponga de una nueva variable de salida que suele denominarse RCO (*Ripple Carry Output*).

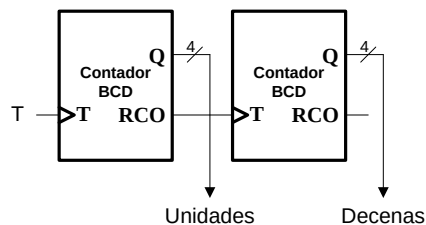


Figura 8.19. Encadenamiento de 2 contadores BCD de manera asíncrona. Cada contador se encarga de dar uno de los 2 dígitos de un número decimal que irá del 00 al 99. El primer contador va incrementado el dígito correspondiente a las unidades a cada flanco de sincronismo externo y dispone de una señal de salida RCO que genera un flanco al segundo contador para que éste se incremente cada vez que el primero reinicia un ciclo de conteo, es decir al pasar de 09 a 10, al pasar de 19 a 20, al pasar de 29 a 30, etc.

La intención es pues que el contador sea capaz de generar un flanco de subida cada vez que pase del último valor del código de conteo al primero, es decir cuando se reinicie como contador cíclico. Una forma simple de generar RCO es simplemente crear un sistema combinacional interno al contador que se ponga a cero cuando llegue al último valor del código de conteo (en el ejemplo en BCD cuando llega a 9) y que se ponga a uno cuando se reinicie y pase al primer valor (en el ejemplo en BCD cuando vuelve a iniciarse en 0).

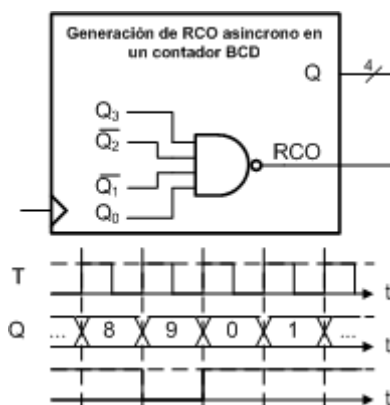


Figura 8.20. Implementación de la señal *RCO* asíncrona en un contador BCD. Vemos que simplemente precisamos de una puerta NAND de 4 entradas que hará que cuando el contador llegue a nueve entonces ponga *RCO* a 0 mientras que para el resto de combinaciones y, por consiguiente, para el paso de 9 a 0, valga uno, generando entonces un flanco de subida que puede utilizarse como flanco de entrada de otro contador y de esta manera permitir el encadenamiento entre ellos.

8.4.2. Encadenamiento de contadores de forma síncrona

En este caso el sincronismo que usan los contadores es el mismo, por lo que se precisa que el contador disponga de una entrada de inhibición (tipo *Enable*) para que se le pueda indicar si “le toca” contar o no según si un contador anterior ha llegado o no al límite de su capacidad de conteo. En el caso de los contadores síncronos esta señal de entrada suele denominarse *RCI* (*Ripple Carry Input*).

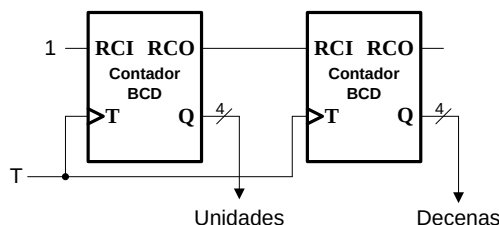


Figura 8.21. Encadenamiento de 2 contadores BCD de manera síncrona. Cada contador dispone de una señal tipo *Enable* denominada *RCI* que cuando vale uno permite que el contador se incremente al llegar un flanco de sincronismo. El contador de las unidades siempre está habilitado para contar pero el contador de las decenas solo se habilitará cuando el primero genere una señal al segundo (*RCO*) indicando a éste que puede contar cuando le llegue el próximo sincronismo. Esto lo hará cuando esté en el último valor de su código de conteo, es decir en el 9.

Para que funcione correctamente el contador ha de poner su salida *RCO* a uno cuando está en el último valor de su código de conteo (en el caso BCD, a 9) puesto que entonces con esta señal habilita al siguiente contador para que cuando a éste último le llegue un sincronismo avance su conteo en una unidad.

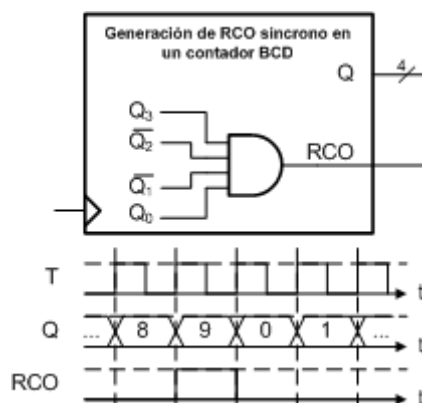


Figura 8.22. Implementación de la señal *RCO* síncrona en un contador BCD. Vemos que simplemente precisamos de una puerta AND de 4 entradas que hará que cuando el contador llegue a nueve entonces ponga *RCO* a 1 habilitando entonces al siguiente contador para que pueda contar cuando le llegue un flanco de entrada.

8.4.3. Contadores comerciales

Los contadores comerciales suelen disponer de estas entradas que permiten el encadenamiento entre ellos para incrementar la capacidad de conteo. A modo de ejemplo en esta sección explicitaremos algunos de los más habituales.

Dispositivo	Descripción
7490	Asíncrono BCD con señal de <i>Reset</i>
7493	Asíncrono binario natural de 4 bits con señal de <i>Reset</i>
74160	Síncrono BCD con señales de <i>Reset</i> y $C/\bar{L}$
74163	Síncrono binario natural de 4 bits con señales de <i>Reset</i> y $C/\bar{L}$
74190	Síncrono BCD con señales de $\bar{U}/D$ y $C/\bar{L}$
74191	Síncrono binario natural de 4 bits con señales de $\bar{U}/D$ y $C/\bar{L}$

Tabla 8.10. Circuitos integrados de la serie 74 que implementan contadores. Todos ellos presentan la opción de encadenamiento, algunos de modo asíncrono y otros de modo síncrono. Vemos que los códigos de conteo habituales en estos contadores son el BCD o bien el binario natural. También es interesante observar que mayoritariamente presentan señales adicionales de control que permiten ponerlos a cero, realizar una carga en paralelo o incluso contar de modo ascendente o descendente. Comentar que sólo se han especificado las características elementales, pero en general disponen de más entradas de control/configuración para cuyo análisis tendríamos que remitirnos al catálogo correspondiente.

La mayoría de aplicaciones comerciales con circuitos integrados discretos suelen poderse realizar mediante cualquiera de estos dispositivos. Os animamos a realizar alguna búsqueda por Internet en la que aparezca alguna aplicación de conteo para poderlo corroborar.

8.5. Contadores en VHDL

El lenguaje VHDL tiene definidos unos tipos de señales especiales y un conjunto de operaciones y funciones asociadas a dichos tipos orientados a la realización de operaciones aritméticas binarias. Dichos tipos son *signed* y *unsigned*. Una señal o variable tipo *signed* es interpretada como un número codificado en complemento a 2, mientras que una tipo *unsigned* es interpretada como un número codificado en binario natural. Existen definidas una serie de operadores aritméticos (+, -, *...) y relacionales (=, <, >=...) que permiten trabajar directamente con señales y variables de estos tipos, así como funciones de conversión para convertir una señal de un tipo a otro (por ejemplo, convertir un vector de bits tipo *std_logic_vector* a tipo *signed* para poder realizar operaciones aritméticas). Todos estos tipos, operadores y funciones están definidos en el paquete “numeric_std” de la librería “ieee”, y se tendrá que declarar su uso si se desean usar estos tipos y operadores:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

Veremos a continuación dos posibles formas de implementar contadores en VHDL.

La primera, usando estos tipos y operadores aritméticos que acabamos de comentar, lo que será útil y cómodo cuando el código de conteo sea binario natural, al poder incrementar o decrementar fácilmente el contador mediante el uso de sentencias del tipo “n <= n + 1” o similares.

La segunda, construyendo directamente el contador mediante la conexión entre un registro EP/SP y un sistema combinacional, tal como se ha explicado a lo largo de este tema.

8.5.1. Contador usando operadores aritméticos

La descripción de un contador binario natural cíclico de 4 bits con reset asíncrono podría ser la siguiente:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Binary_counter_4bit is
    port (
        Clk, Reset    : in    std_logic;
        Q              : out   std_logic_vector(3 downto 0));
end Binary_counter_4bit;

architecture arithmetic of Binary_counter_4bit is
    signal Q_reg, Q_next : unsigned(3 down to 0);
```



```
begin

process(Clk, Reset) is

begin
    if Reset = '1' then
        Q_reg <= (others => '0');
    elsif (Clk'event and Clk = '1') then
        Q_reg <= Q_next;
    end if;

end process;

Q_next <= Q_reg + 1;

Q <= std_logic_vector(Q_reg);

end arithmetic;
```

El esquema de esta descripción es como la usada en el tema anterior para los registros, y sólo conviene detenerse en tres líneas especiales del código. En primer lugar, mediante la línea

```
signal Q_reg, Q_next      : unsigned(3 down to 0);
```

definimos dos señales intermedias de 4 bits de tipo *unsigned*, para poder utilizar el operador aritmético “+”, que no podríamos utilizar directamente sobre señales de tipo *std_logic_vector*. En segundo lugar, en la asignación

```
Q_next <= Q_reg + 1;
```

Usamos el operador aritmético “+” para incrementar en una unidad el valor del contador. Por último, dado que la señal *Q_reg* es de tipo *unsigned* y nuestro puerto de salida *Q* es de tipo *std_logic_vector*, necesitamos usar la función de conversión adecuada (de *unsigned* a *std_logic_vector*) para poder asignar el valor de la señal *Q_reg* al puerto de salida del contador *Q*:

```
Q <= std_logic_vector(Q_reg);
```

Observemos como ha implementado el Quartus Prime este contador:

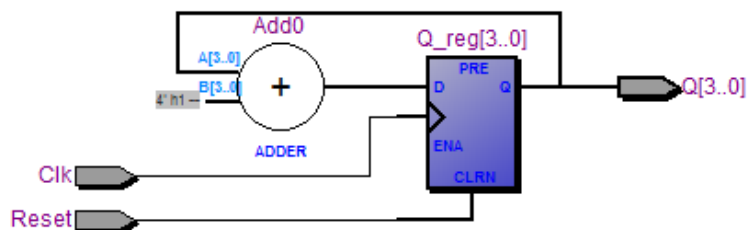


Figura 8.23. Vista RTL del contador de 4 bits descrito según el código VHDL propuesto.

El resultado de la simulación de este contador nos da el siguiente diagrama de tiempos:

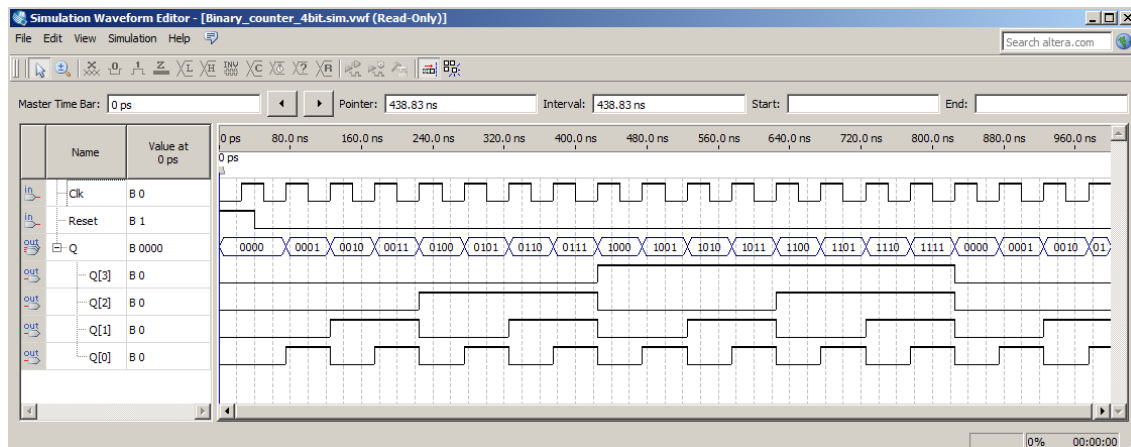


Figura 8.24. Simulación del contador de 4 bits descrito según el código VHDL propuesto.

Como se puede observar, la salida del contador se va incrementando en cada flanco de subida de la señal de sincronismo, y al estar definidas las señales como vectores de 4 bits, después de la combinación “1111” vuelve a comenzar de nuevo desde la “0000”.

8.5.2. Contador usando registro y sistema combinacional

Realizaremos ahora la descripción del mismo contador anterior, pero sin usar los tipos y operadores aritméticos de VHDL. La estructura será la misma en cuanto al registro EP/SP, pero sustituiremos el “sumador” por un sistema combinacional que implementa el código de conteo del contador:

```
library ieee;
use ieee.std_logic_1164.all;

entity Binary_counter_4bit_2 is
    port
    (
        Clk, Reset    : in    std_logic;
        Q              : out   std_logic_vector(3 downto 0)
    );
end Binary_counter_4bit_2;

architecture logic of Binary_counter_4bit_2 is
    signal Q_reg, Q_next : std_logic_vector(3 downto 0);

begin
```

```

process(Clk, Reset) is
begin
    if Reset = '1' then
        Q_reg <= (others => '0');
    elsif (Clk'event and Clk = '1') then
        Q_reg <= Q_next;
    end if;

end process;

Q_next <=
    "0001" when Q_reg = "0000" else
    "0010" when Q_reg = "0001" else
    "0011" when Q_reg = "0010" else
    "0100" when Q_reg = "0011" else
    "0101" when Q_reg = "0100" else
    "0110" when Q_reg = "0101" else
    "0111" when Q_reg = "0110" else
    "1000" when Q_reg = "0111" else
    "1001" when Q_reg = "1000" else
    "1010" when Q_reg = "1001" else
    "1011" when Q_reg = "1010" else
    "1100" when Q_reg = "1011" else
    "1101" when Q_reg = "1100" else
    "1110" when Q_reg = "1101" else
    "1111" when Q_reg = "1110" else
    "0000";

Q <= Q_reg;

end logic;

```

El resultado de la simulación de este contador nos da el siguiente diagrama de tiempos:

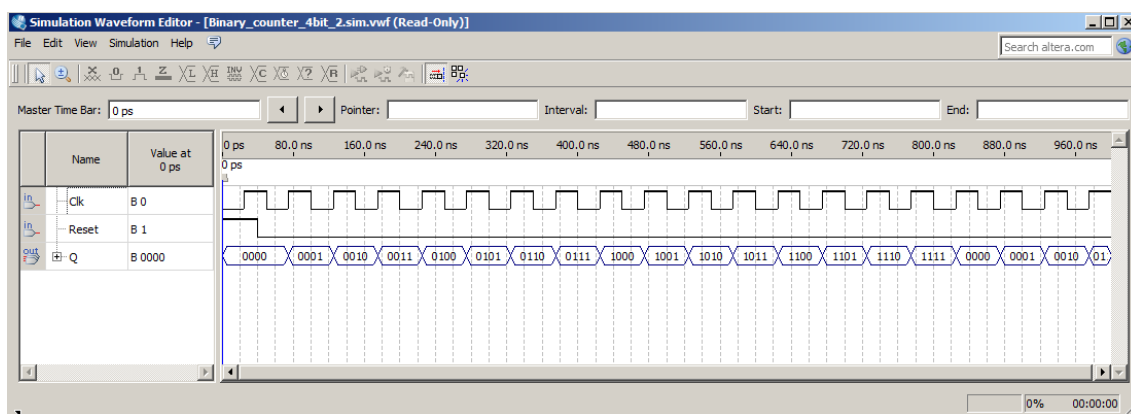


Figura 8.25. Simulación del contador de 4 bits descrito según el código VHDL propuesto.

Como se puede observar, el diagrama de tiempos obtenido es idéntico al del anterior contador.

Como curiosidad, observemos cómo ha implementado el Quartus Prime este contador:

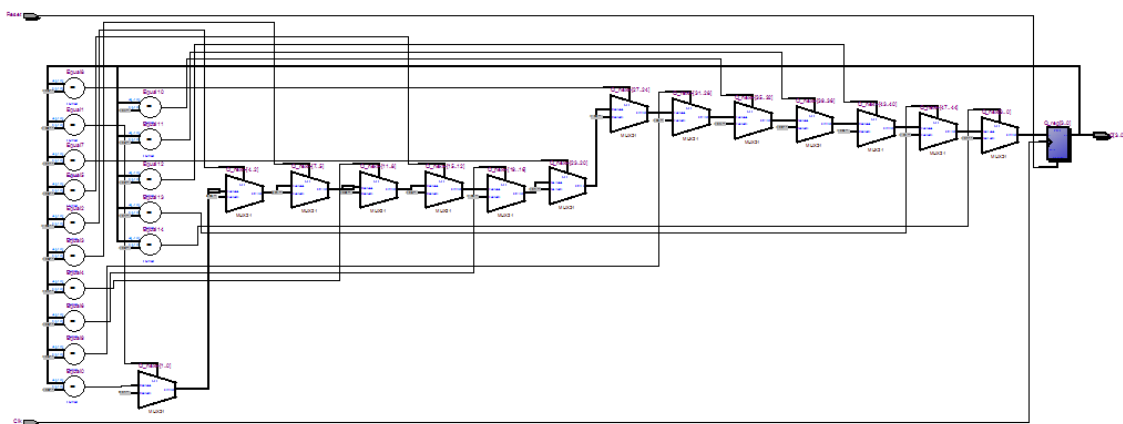


Figura 8.26. Vista RTL del contador de 4 bits descrito usando una sentencia *conditional signal assignment* para la obtención de Q_next.

La implementación del sistema combinacional que implementa el código de conteo se realiza mediante la concatenación de múltiples etapas de multiplexores propia de la sentencia de asignación condicional de valores (*conditional signal assignment*) usada para la obtención de la señal Q_next. Esto podría introducir un retardo importante en la obtención de la señal Q_next que limitaría la frecuencia máxima de funcionamiento del contador. Podríamos probar a cambiar esta asignación por una de tipo de selección (*selected signal assignment*) como la siguiente:

```
with Q_reg select
  Q_next <=
    "0001" when "0000",
    "0010" when "0001",
    "0011" when "0010",
    "0100" when "0011",
    "0101" when "0100",
    "0110" when "0101",
    "0111" when "0110",
    "1000" when "0111",
    "1001" when "1000",
    "1010" when "1001",
    "1011" when "1010",
    "1100" when "1011",
    "1101" when "1100",
    "1110" when "1101",
    "1111" when "1110",
    "0000" when others;
```

Observemos cómo ha implementado ahora el Quartus Prime el sistema combinacional:

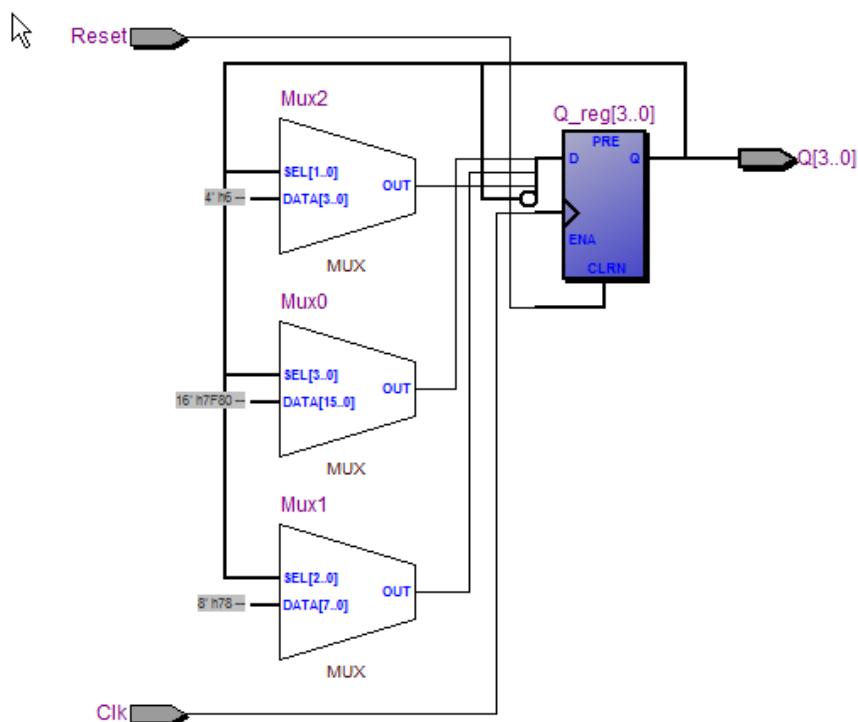


Figura 8.27. Vista RTL del contador de 4 bits descrito usando una *selected signal assignment* para la obtención de Q_next .

Observamos que ahora el sistema combinacional se implementa mediante una única etapa de multiplexores, lo que seguramente comportará un retardo menor en la obtención de la señal Q_next .

Se deja al lector que compruebe que el diagrama de tiempos obtenido mediante la simulación de este contador es idéntico a las obtenidas anteriormente.